

Grau en Matemàtica Computacional i Analítica de Dades

UNIVERSITAT AUTÒNOMA DE BARCELONA



**PHYSICS-INFORMED NEURAL NETWORKS
I LES SEVES APLICACIONS**

TREBALL DE FI DE GRAU

**Lluís Panal Majó
1668072**

Supervisat per Sílvia Cuadrado Gavilán i Alejandra Cabaña Nigro

Juny 2026

Resum

L'objecte d'estudi d'aquest treball són les *Physics-Informed Neural Networks* (PINNs) i les seves aplicacions en la resolució d'equacions diferencials. Primer, es presenta la fonamentació teòrica d'aquestes xarxes neuronals, detallant l'ús de la diferenciació automàtica i com s'incorporen les lleis físiques (EDO i EDP) dins de la funció de pèrdua. A continuació, es classifiquen les seves tipologies i es fa una comparativa amb els mètodes numèrics tradicionals, posant de manifest els avantatges i els reptes computacionals que presenten. Seguidament, el model es posa a prova i es valida resolent problemes analítics i numèrics clàssics: l'equació logística, l'equació de Burgers i l'equació de Fisher. Finalment, s'estudia la inestabilitat de Turing mitjançant el model de Meinhardt per analitzar el comportament de les PINNs en problemes més complexos. Es posa a prova de dues maneres: el problema directe, on es busca captar i simular la formació de patrons, i el problema invers, on un dels objectius és trobar el valor dels paràmetres físics a partir de dades.

Abstract

This research focuses on Physics-Informed Neural Networks (PINNs) and their applications in solving differential equations. First, the theoretical foundation of these neural networks is presented, detailing the use of automatic differentiation and how physical laws (ODEs and PDEs) are incorporated into the loss function. Next, their typologies are classified, and a comparison with traditional numerical methods is conducted, highlighting their advantages and computational challenges. Then, the model is tested and validated by solving classic analytical and numerical problems: logistic equation, Burgers' equation, and Fisher's equation. Finally, Turing instability is studied through the Meinhardt model to analyze the behavior of PINNs in more complex problems. It is tested in two ways: the forward problem, aiming to capture and simulate pattern formation, and the inverse problem, where one of the objectives is to find the value of physical parameters from data.

Índex

1	Motivació i objectius	4
2	Fonamentació Teòrica	5
2.1	Xarxes Neuronals i Diferenciació Automàtica	5
2.2	Incorporació de lleis físiques (EDO o EDP) en la funció de pèrdua	7
2.3	El segon ús de la diferenciació automàtica per a les PINNs	8
3	Tipologies de PINNs: Models continus i discrets	9
3.1	Models de temps continu	9
3.2	Models de temps discret	9
4	Comparativa PINN i mètodes tradicionals	11
4.1	Avantatges de les PINNs	11
4.2	Inconvenients i Reptes de les PINNs	11
5	Proves i Validació de les PINNs	13
5.1	Equació Logística	13
5.2	Equació de Burgers	15
5.3	Equació de Fisher	16
6	La Inestabilitat de Turing	19
6.1	Fonament Matemàtic	19
6.2	Aplicació a un sistema concret: El Model de Meinhardt	23
6.2.1	Resultats de la PINN	25
6.2.2	El problema invers en el model de Meinhardt	27
7	Conclusions	30
8	Apèndix	31
8.1	Resolució de l'equació de Burgers mitjançant el mètode de les característiques . .	31
8.2	Codi	33

1 Motivació i objectius

Quan va arribar el moment d'escollir el tema del Treball de Fi de Grau, no tenia del tot clar quin seria l'enfocament exacte. El que sí que tenia era alguna preferència: d'una banda, durant la carrera m'havien agradat molt les assignatures d'Equacions Diferencials Ordinàries i Equacions en Derivades Parcial; de l'altra, trobava molt interessant el món del *Machine Learning* i el *Deep Learning*, també vist en assignatures del grau.

Amb aquesta idea al cap, vaig decidir contactar amb la Sílvia Cuadrado, que havia estat la meva professora d'EDOs i EDPs. Li vaig explicar la meua voluntat d'ajuntar l'estudi de les equacions amb la intel·ligència artificial, i ella em va presentar les *Physics-Informed Neural Networks* (PINNs). Fins aquell moment no sabia què eren, però em va impressionar molt el fet que el model pogués aprendre a predir la solució d'una equació sense tenir punts reals de dades, guiant-se exclusivament pel compliment de l'equació. A més, l'Alejandra Cabaña va veure el tema, li va agradar i va decidir apuntar-s'hi com a cotutora.

Per tal de portar aquestes xarxes a una aplicació pràctica, la Sílvia em va proposar treballar amb la inestabilitat de Turing. És un fenomen matemàtic i biològic molt interessant que prové dels treballs d'Alan Turing de l'any 1952 [6].

A partir d'aquí, es van establir els següents objectius per al treball:

- Fonamentació teòrica: Comprendre com s'incorporen les lleis físiques (EDO o EDP) en la funció de pèrdua i entendre l'ús addicional de la diferenciació automàtica.
- Analitzar i descriure els diferents tipus de PINNs.
- Analitzar els avantatges i limitacions d'aquestes xarxes respecte als mètodes numèrics tradicionals.
- Implementació i validació en models senzills: Construir el model de xarxa neuronal i validar-ne el correcte funcionament aplicant-lo primer a equacions clàssiques i conegudes.
- Estudiar la inestabilitat de Turing i aplicar les PINNs al model de Meinhardt per tal de resoldre el problema tant de manera directa com inversa.

2 Fonamentació Teòrica

En aquest apartat es desenvoluparan els conceptes matemàtics i computacionals essencials que defineixen les *Physics-Informed Neural Networks* (PINNs). L'objectiu principal d'aquesta secció és comprendre com les xarxes neuronals poden ser forçades a seguir les lleis físiques indicades per Equacions Diferencials Ordinàries (EDO) o Equacions en Derivades Parcial (EDP). Aquest esquema és un nou mètode per a aproximar la solució de les equacions aprofitant la capacitat de les xarxes neuronals.

Això s'aconsegueix mitjançant dos mecanismes fonamentals que s'analitzaran a continuació: la incorporació del residu de l'equació diferencial (que, tot i que s'explicarà amb més detall més endavant, quantifica l'error de la xarxa respecte a les equacions) dins de la funció de pèrdua i l'ús de la diferenciació automàtica per calcular les derivades exactes de la sortida de la xarxa respecte a les seves variables d'entrada.

2.1 Xarxes Neuronals i Diferenciació Automàtica

Una xarxa neuronal és un model matemàtic que s'utilitza per aproximar funcions no lineals. Matemàticament, es defineix com una composició seqüencial de múltiples operacions lineals i no lineals organitzades en estructures anomenades capes: una capa d'entrada, una o diverses capes intermèdies (capes ocultes), i una capa de sortida.

Cada capa està formada per unitats anomenades "neurons". Cada neurona rep un vector d'entrades $x = (x_1, x_2, \dots, x_n)$, on cadascuna d'elles es multiplica per un paràmetre anomenat "pes" (W_i). A la suma d'aquests productes s'hi afegeix un terme independent, el "biaix" (b). Aquestes operacions produeixen un valor intermedi z :

$$z = \sum_{i=1}^n W_i x_i + b = W^T x + b. \quad (1)$$

Finalment, aquest resultat intermedi s'avalua en una funció d'activació no lineal, que denotarem com $\sigma(\cdot)$ (com ara la tangent hiperbòlica o la funció Sigmoide), per obtenir la sortida de la neurona a :

$$a = \sigma(z) = \sigma(W^T x + b). \quad (2)$$

L'ús d'aquestes funcions no lineals és fonamental, ja que sense elles, la xarxa sencera equivaldria a una simple regressió lineal i no podria modelar les funcions no lineals. El conjunt global de tots els pesos i biaixos de la xarxa conformen els seus paràmetres, θ , que s'ajusten en l'etapa d'entrenament.

Les xarxes neuronals aprenen a partir de dades que veu el model. S'entrenen minimitzant una funció de pèrdua, $\mathcal{L}(\theta)$, que mesura l'error entre les prediccions del model, $u_\theta(x_i)$, i les dades reals observades de les quals disposem, y_i . L'elecció d'aquesta funció depèn de la naturalesa del problema. Per a problemes de regressió on s'aproximen funcions contínues, com és el cas de les

PINN, sovint s'utilitza l'Error Quadràtic Mitjà (MSE):

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N |u_{\theta}(x_i) - y_i|^2. \quad (3)$$

Per minimitzar aquest error i actualitzar els paràmetres iterativament, s'utilitzen algorismes d'optimització (com el descens del gradient). Aquests algorismes fan evolucionar els paràmetres θ avançant en la direcció oposada al gradient de la funció de pèrdua. Sigui η la taxa d'aprenentatge (*learning rate*), l'actualització es defineix de la següent manera:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}. \quad (4)$$

Per poder calcular aquest gradient $\nabla_{\theta} \mathcal{L}$, cal avaluar com varia l'error respecte a cadascun dels paràmetres interns. El mètode utilitzat és la diferenciació automàtica (AD, per les seves sigles en anglès) en el seu mode invers (és a dir, des de la sortida), coneguda en l'aprenentatge automàtic com a retropropagació o *backpropagation*.

L'AD es basa en un principi fonamental: per molt complexa que sigui la xarxa neuronal, totes les seves matemàtiques es poden descompondre en una seqüència de sumes, multiplicacions i funcions elementals simples, de les quals és fàcil calcular la derivada. Quan la xarxa agafa unes dades d'entrada i fa els càlculs per generar una predicció (*forward pass*), es crea internament un graf computacional, que és com un mapa que registra pas a pas l'ordre de cada operació matemàtica.

Com que la derivada de cada operació és coneguda per l'ordinador, per trobar la derivada de tota la xarxa només cal recórrer aquest mapa en sentit invers: començant per l'error final $\mathcal{L}$ i retrocedint cap a les entrades, aplicant la regla de la cadena.

Per trobar com afecta un pes concret W_i de l'última capa a l'error total, l'algorisme encadena la derivada de les operacions intermèdies (1) i (2) que ja estan registrades al graf:

$$\frac{\partial \mathcal{L}}{\partial W_i} = \frac{\partial \mathcal{L}}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial W_i}. \quad (5)$$

Per actualitzar els pesos d'una capa oculta qualsevol l , l'algorisme no necessita recalculer les derivades des de la sortida fins a aquella capa, sinó que propaga l'error ja obtingut a la capa immediatament posterior $l + 1$.

Si definim el terme d'error d'una capa com la derivada de la funció de pèrdua respecte a la seva combinació lineal abans de l'activació, $\delta^{(l)} = \frac{\partial \mathcal{L}}{\partial z^{(l)}}$, la regla de la cadena ens permet relacionar capes adjacents amb la següent expressió:

$$\delta^{(l)} = \left(\left(W^{(l+1)} \right)^T \delta^{(l+1)} \right) \odot \sigma' \left(z^{(l)} \right). \quad (6)$$

En aquesta equació, $(W^{(l+1)})^T$ és la matriu de pesos transposada de la capa següent, $\delta^{(l+1)}$

és l'error que prové d'aquesta capa posterior, $\sigma'(z^{(l)})$ és la derivada de la funció d'activació avaluada en la capa actual l , i el símbol $\odot$ representa el producte de matrius element a element (*Hadamard product*).

Un cop obtingut aquest terme $\delta^{(l)}$ de forma eficient, el gradient respecte a qualsevol matriu de pesos d'aquella mateixa capa s'obté directament multiplicant l'error per la sortida de la capa anterior:

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \delta^{(l)} \left(a^{(l-1)} \right)^T, \quad (7)$$

$$\frac{\partial \mathcal{L}}{\partial b^{(l)}} = \delta^{(l)}. \quad (8)$$

D'aquesta manera, amb un únic recorregut invers pel graf, l'AD és capaç de calcular simultàniament les derivades respecte a tots els paràmetres del model de forma exacta, fent possible l'entrenament de xarxes amb milions de variables.

2.2 Incorporació de lleis físiques (EDO o EDP) en la funció de pèrdua

En problemes de l'enginyeria i la física, sovint coneixem per endavant les regles que el sistema ha de seguir, les quals s'acostumen a expressar matemàticament mitjançant Equacions Diferencials Ordinàries (EDO) o Equacions en Derivades Parcial (EDP). Les PINNs aprofiten aquest coneixement teòric introduint-lo en la funció de pèrdua de la xarxa neuronal com un terme addicional.

Imaginem una equació diferencial que descriu un fenomen (com la propagació de la calor o el moviment d'un fluid). Aquesta equació ens descriu el comportament d'una variable (en molts casos una densitat) que ens interessa conèixer, diguem-ne u (per exemple, la temperatura), que depèn de l'espai $x \in \mathbb{R}^n$ i del temps t .

El primer pas de les PINNs consisteix a agrupar tots els termes de la nostra equació diferencial a un sol costat de la igualtat, de manera que a l'altre costat només hi quedi un zero. De forma general, aquest procés es representa amb la següent notació:

$$\mathcal{N}[u(x, t)] = 0. \quad (9)$$

En aquesta expressió, el símbol $\mathcal{N}[\cdot]$ engloba tots els elements de l'equació diferencial (derivades temporals, derivades espacials, constants, etc.). El significat d'aquest pas és que si $u(x, t)$ és solució de l'equació, aleshores es compleix que $\mathcal{N}[u(x, t)] = 0$ a tot el domini.

Una PINN utilitza una xarxa neuronal, que anomenarem $u_\theta(x, t)$, per aproximar aquesta solució. Aquesta xarxa aconsegueix fer una bona aproximació, ja que s'entrena penalitzant-la si no compleix la física del problema. Això es fa definint l'error total o funció de pèrdua, $\mathcal{L}(\theta)$, com la suma de dues parts principals:

$$\mathcal{L}(\theta) = \mathcal{L}_{data}(\theta) + \mathcal{L}_{física}(\theta). \quad (10)$$

El primer terme, $\mathcal{L}_{data}(\theta)$, és l'error clàssic basat en dades. Mesura l'Error Quadràtic Mitjà (MSE) entre les prediccions que fa la xarxa i els valors reals que coneixem prèviament, que tant

pot ser les mesures experimentals preses com les condicions inicials i de contorn:

$$\mathcal{L}_{data}(\theta) = \frac{1}{N_u} \sum_{i=1}^{N_u} |u_\theta(x_i, t_i) - u_i|^2, \quad (11)$$

on N_u és el nombre de punts etiquetats de les condicions inicials i de contorn i $\{x_i, t_i, u_i\}_{i=1}^{N_u}$ és el conjunt de dades on coneixem el valor exacte de la solució u_i per a cada coordenada espai-temporal.

El segon terme, $\mathcal{L}_{física}(\theta)$, és l'anomenat residu físic o terme de pèrdua basat en la física. Aquest terme és el que permet incorporar l'EDO o l'EDP a la xarxa. Es calcula avaluant el nostre operador $\mathcal{N}$ sobre les prediccions de la xarxa en un conjunt de punts arbitraris de l'espai i el temps:

$$\mathcal{L}_{física}(\theta) = \frac{1}{N_f} \sum_{j=1}^{N_f} |\mathcal{N}[u_\theta(x_j, t_j)]|^2, \quad (12)$$

on N_f és el nombre de punts de col·locació distribuïts dins del domini i $\{x_j, t_j\}_{j=1}^{N_f}$ són aquests punts.

Si la xarxa neuronal prediu una solució que trenca les lleis de la física, l'avaluació de l'operador $\mathcal{N}$ no donarà zero. Aquest excés obliga la xarxa a ajustar els seus pesos per fer que aquest valor sigui el més proper possible a zero. Per aquest motiu, aquest mecanisme ens garanteix que el model prediu resultats que tenen un sentit físic real. És a dir, tot i que no coneixem quant val $u(x_j, t_j)$, sabem que $\mathcal{N}[u_\theta(x_j, t_j)]$ ha de valdre 0 a tot el domini, la qual cosa redueix significativament la quantitat de dades necessàries per entrenar el model.

2.3 El segon ús de la diferenciació automàtica per a les PINNs

La diferència principal d'una PINN respecte a una altra xarxa neuronal és que utilitza la diferenciació automàtica en dues ocasions.

- Igual que qualsevol altra xarxa neuronal, utilitzen l'AD per a optimitzar els paràmetres.
- D'altra banda, per tal de poder avaluar l'equació diferencial i calcular el residu físic, necessitem obtenir les derivades de la funció que aproxima la xarxa respecte a les seves variables d'entrada. Les PINNs utilitzen l'AD en el seu mode directe, que ens permet derivar la predicció del model respecte a aquestes variables de forma contínua, obtenint resultats exactes on l'únic error existent és el de la precisió de màquina de l'ordinador. Així podem prescindir completament dels mètodes numèrics tradicionals que requereixen crear malles discretes de punts i fer aproximacions que podien introduir errors.

3 Tipologies de PINNs: Models continus i discrets

Depenent de com es tracti la variable temporal i de com s'estructuri l'arquitectura de la xarxa per resoldre l'equació diferencial, les PINNs es poden classificar principalment en dos enfocaments: els models de temps continu i els models de temps discret.

3.1 Models de temps continu

L'enfocament continu és la formulació més intuïtiva. En aquest model, les variables d'entrada són els temps i la posició, i la sortida seria el valor de la funció en aquell punt. És a dir, la xarxa neuronal és una funció $u_\theta(x, t)$ que intenta aproximar la solució en qualsevol punt del domini espai-temps.

El principal avantatge és que no cal definir una quadrícula temporal fixa; podem avaluar la solució en qualsevol instant de temps dins del domini d'entrenament. Les dades d'entrenament poden ser només dades donades per les condicions inicials o de frontera. El residu de la física es calcula en un conjunt de punts distribuïts en tot el domini, sense necessitat de saber el valor de la funció corresponent.

Per altra banda, en problemes amb una evolució temporal molt complexa o caòtica, la xarxa pot tenir dificultats per convergir, ja que ha d'aprendre tota la trajectòria del sistema d'un sol cop.

3.2 Models de temps discret

A diferència de l'enfocament continu, els models de temps discret no tracten el temps com una variable d'entrada, sinó que integren esquemes de discretització clàssics de l'anàlisi numèrica, com els mètodes de Runge-Kutta (RK), directament en l'arquitectura de la xarxa neuronal. Aquest enfocament és especialment útil quan es disposa de dades en instants temporals concrets i es vol garantir una propagació de la solució amb un ordre de precisió elevat.

En lloc de predir $u(x, t)$ per tot el domini, la xarxa neuronal s'entrena per aprendre l'operador de transició entre dos instants de temps, t_n i t_{n+1} , separats per un pas temporal Δt . La sortida de la xarxa no és un únic valor, sinó que es ramifica en múltiples sortides que representen les q etapes intermèdies del mètode numèric. En el cas del mètode Runge-Kutta:

$$u_{n+c_i} = u_n + \Delta t \sum_{j=1}^q a_{ij} \mathcal{N}[u_{n+c_j}], \quad i = 1, \dots, q. \quad (13)$$

L'ús d'un mètode tradicional a la sortida permet utilitzar passos temporals (Δt) relativament grans sense perdre precisió, ja que es poden emprar esquemes de moltes etapes que requeririen de molts recursos computacionals en mètodes clàssics, però que són més eficients de processar per a una xarxa neuronal.

En aquest cas es necessitarien dades del punt inicial, però també del punt final. La funció de pèrdua que s'utilitza és diferent de la que hem vist anteriorment, ja que, tot i que seguim imposant que compleixi les condicions de frontera i, si se'n disposa, les mesures experimentals,

també volem imposar que compleixi els passos del mètode numèric. Per tant, la funció de pèrdua es construeix sumant aquests dos conceptes:

$$\mathcal{L}(\theta) = \mathcal{L}_{residu}(\theta) + \mathcal{L}_{data}(\theta). \quad (14)$$

On el terme del residu físic ara inclou el sumatori sobre tots els punts de dades $\{x_i^n, u_i^n\}_{i=1}^{N_n}$ disponibles en l'instant t_n :

$$\mathcal{L}_{residu}(\theta) = \frac{1}{N_n} \sum_{i=1}^{N_n} \sum_{j=1}^{q+1} |u_i^n - u_{i,j}^n|^2. \quad (15)$$

Aquí, u_i^n és el valor real conegut i $u_{i,j}^n$ són els valors que la xarxa diu que "haurien de valdre" en el passat, al pas corresponent del mètode numèric, per ser coherents amb la física de l'equació.

Finalment, s'afegeix el terme de les condicions de frontera per garantir que la solució respecti els límits del domini espacial en cada etapa i per tal que compleixi les mesures experimentals:

$$\mathcal{L}_{data}(\theta) = \frac{1}{N_u} \sum_{i=1}^{N_u} |u_\theta(x_i, t_i) - u_i|^2. \quad (16)$$

Al llarg del treball s'utilitzaran només els models continus, però és important saber que existeix aquesta altra modalitat.

4 Comparativa PINN i mètodes tradicionals

La resolució d'equacions diferencials, quan no és possible fer-ho de forma analítica, ha estat dominada històricament per mètodes numèrics clàssics com el Mètode de Diferències Finites (FDM) o el Mètode d'Elements Finites (FEM). L'aparició de les PINNs introdueix un nou paradigma que presenta tant avantatges significatius com nous reptes computacionals. A continuació s'exposa una comparativa entre ambdós enfocaments.

4.1 Avantatges de les PINNs

- **Mètode sense malla i resistència a la dimensionalitat:** No requereixen la generació d'una malla computacional per discretitzar el domini espacial. En els mètodes tradicionals, aquest procés pot ser extremadament costós, ja que el nombre de nodes de la malla creix de forma exponencial a mesura que augmenten les dimensions del problema. En canvi, en les PINNs el cost computacional depèn simplement del nombre de punts arbitraris escollits per a l'entrenament (punts de col·locació per al residu de l'equació, punts per a les condicions inicials (IC) i de contorn (BC) i, en alguns casos, punts de mesures experimentals). Això els atorga una capacitat d'escalat molt superior i mitiga aquest problema.
- **Capacitat per a resoldre problemes inversos:** Donat un nombre de punts de la solució, són capaces de trobar la funció i els seus paràmetres desconeguts. Veure 6.2.2 on s'aprofita aquesta capacitat.
- **Avaluació ràpida:** Un cop el model ha estat entrenat i els pesos han estat optimitzats, es pot avaluar qualsevol punt del domini de forma pràcticament instantània. Això es deu al fet que la inferència es redueix a una simple multiplicació de matrius i avaluació de funcions d'activació a través de la xarxa neuronal (*forward pass*), la qual cosa és molt més ràpida que els càlculs iteratius dels integradors numèrics clàssics.
- **Aprenentatge per transferència (*Transfer Learning*):** El temps d'entrenament es redueix dràsticament inicialitzant la xarxa amb els pesos d'una altra PINN prèviament entrenada per a problemes similars.

4.2 Inconvenients i Reptes de les PINNs

- **Entrenament lent:** Resoldre un problema directe des de zero requereix milers d'èpoques. És un procés computacionalment molt exigent; de fet, durant la realització d'aquest treball, la fase d'entrenament d'algun dels models ha trigat més de 20 minuts a completar-se.
- **Manca de garanties teòriques de convergència i fites d'error:** A diferència dels mètodes clàssics, l'entrenament d'una PINN és, en essència, un problema d'optimització no convex. Això fa que la xarxa es pugui quedar atrapada en mínims locals de la funció de pèrdua, sense poder garantir teòricament que convergirà cap al mínim global ni permetent predir el marge d'error *a priori*. Veure 5.1 on es donarà aquesta situació.
- **Dificultat d'aprendre els detalls (F-Principle):** Aprenen ràpidament comportaments globals (baixes freqüències), però tenen greus dificultats per recuperar oscil·lacions petites o canvis bruscos (altes freqüències), com ara discontinuïtats geomètriques.

- **Cerca d'hiperparàmetres:** Mentre que els mètodes tradicionals utilitzen procediments sistemàtics per millorar la resolució espacial o temporal, les PINNs requereixen nombroses proves i assajos per ajustar manualment paràmetres com el nombre de capes, neurones o la taxa d'aprenentatge. Aquest treball s'ha centrat a obtenir uns resultats prou satisfactoris, no en la cerca de la millor combinació d'hiperparàmetres que permetria obtenir el resultat òptim.

5 Proves i Validació de les PINNs

Abans d'abordar sistemes d'equacions més complexes, és important de provar si les PINNs funcionen correctament en problemes més senzills dels quals podem trobar la solució explícita.

5.1 Equació Logística

El primer cas d'estudi és l'equació logística, una equació diferencial ordinària (EDO) no lineal clàssica que modela el creixement d'una població sota recursos limitats. S'expressa de la següent manera:

$$\frac{du}{dt} = f(u) = au(1 - u), \quad (17)$$

on $u(t)$ representa la població en l'instant de temps t , i a determina la taxa de creixement del sistema. Per a la realització d'aquesta prova, s'ha fixat el paràmetre $a = 1.0$ amb un domini temporal $t \in [0, 10]$ i una condició inicial $u(0) = u_0 = 0.1$.

La solució explícita de l'equació és la següent:

$$u(t) = \frac{1}{1 + \left(\frac{1-u_0}{u_0}\right) e^{-at}} = \frac{1}{1 + \frac{1-0.1}{0.1} e^{-t}} = \frac{1}{1 + 9e^{-t}}. \quad (18)$$

A la Figura 1 es pot observar l'evolució d'aquesta solució exacta al llarg del temps

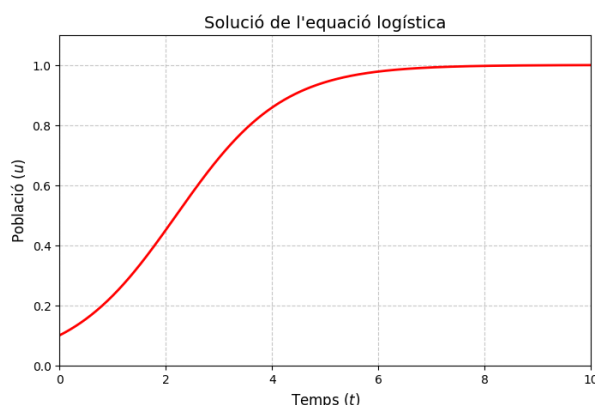


Figura 1: Solució exacta de l'equació logística per $a = 1.0$ i $u_0 = 0.1$ en $t \in [0, 10]$.

Dinàmica del sistema i punts d'equilibri

L'estudi del diagrama de fase sobre la línia real ens permet predir el comportament del sistema identificant els punts d'equilibri, estats estacionaris on el sistema es manté constant en el temps perquè la seva derivada temporal s'anul·la ($f(u) = 0$), que en aquest cas són $u_1^* = 0$ i $u_2^* = 1$. Per determinar l'estabilitat d'aquests equilibris, analitzem el signe del flux $f(u)$ en els intervals delimitats per aquests punts:

- $u < 0$: El flux és negatiu ($f(u) < 0$), fent que les solucions s'allunyin de l'origen cap a valors més negatius.
- $0 < u < 1$: El flux és positiu ($f(u) > 0$), el que implica que qualsevol solució en aquest

rang creixerà, allunyant-se de l'origen i apropant-se al 1.

- $u > 1$: El flux és negatiu ($f(u) < 0$), de manera que les solucions decreixen.

Aquesta anàlisi indica que $u_1^* = 0$ és un equilibri inestable (repulsor), mentre que $u_2^* = 1$ és un equilibri estable (atractor).

Resultats de la PINN

S'ha utilitzat una arquitectura de 3 capes ocultes amb 20 neurones i activació tanh (tangent hiperbòlica). El model es va entrenar amb un pes de 100 per la condició inicial. Tal com es pot observar a la Figura 2, tot i que el model aconsegueix capturar correctament la condició inicial de 0.1, la dinàmica temporal que aprèn és errònia: en lloc de créixer cap a l'equilibri estable, la predicció decau ràpidament cap a l'estat d'extinció $u(t) = 0$. Aquest comportament es deu al fet que el model cau en un mínim local de la funció de pèrdua. Com que el residu físic s'anul·la perfectament a tot el domini quan prediu $u = 0$, alterar la sortida per a treure uns pocs punts d'allà faria augmentar la funció de pèrdua.

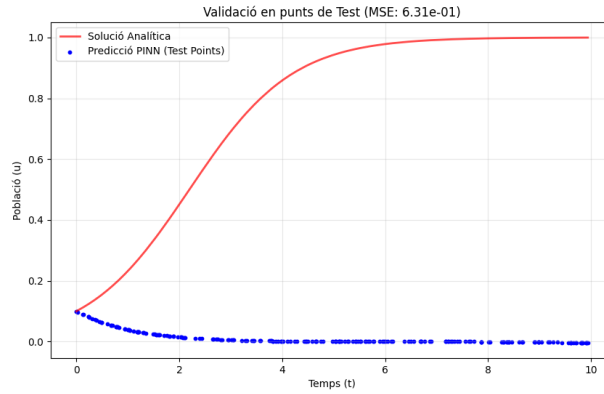


Figura 2: El model respecta la condició inicial, però ignora la tendència logística i s'estabilitza erròniament en l'estat d'extinció ($u = 0$).

Per solucionar aquest problema de forma, s'ha implementat una xarxa a complir $u(0) = u_0$ per construcció i que aprengui a partir d'aquest punt. La sortida de la xarxa es redefineix com:

$$u_{pred}(t) = u_0 + t \cdot u_{\theta}(t), \quad (19)$$

on $u_{\theta}(t)$ és la sortida del model base. Aquesta modificació aporta dos beneficis directes a l'entrenament, traient el model d'aquest mínim local:

- D'una banda, s'elimina la necessitat d'incloure l'error de la condició inicial a la funció de pèrdua, simplificant l'optimització.
- D'altra banda, la xarxa neuronal ($u_{\theta}(t)$) ja no ha de predir directament la població, sinó que passa a modelar la taxa de canvi de la funció. Això modifica l'escala dels valors de sortida i facilita la convergència de l'algorisme cap a la solució correcta.

Aquesta transformació garanteix que el terme temporal s'anul·li a $t = 0$, obligant la xarxa a

explorar la branca de la solució que condueix a l'equilibri estable $u = 1$. Com es mostra a la Figura 3, amb aquesta modificació el model assoleix una precisió molt elevada.

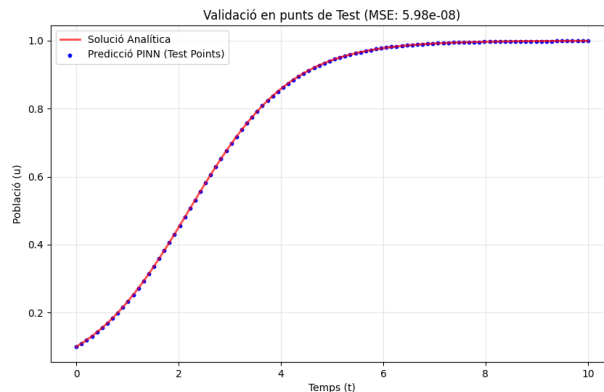


Figura 3: Validació final de la PINN amb la condició inicial imposada de forma estricta. La predicció se solapa perfectament amb la solució analítica.

Aquest ha estat l'únic cas on ha succeït aquest fenomen i ha resultat necessari utilitzar aquesta arquitectura. Això es deu, probablement, al fet que la condició inicial era un únic punt i no hi havia condicions de frontera que assegurassin la solució a l'espai, a diferència de les equacions en derivades parcials (EDP) estudiades posteriorment, on un volum major de dades inicials i de contorn milloren l'optimització.

5.2 Equació de Burgers

El següent cas d'estudi és l'equació de Burgers en la seva variant no viscosa, una equació en derivades parcials (EDP) de primer ordre que serveix com a model per estudiar la formació d'ones de xoc i de rarefacció en dinàmica de fluids. S'expressa de la següent manera:

$$u_t + uu_x = 0. \quad (20)$$

Per a la realització d'aquesta prova, s'ha considerat un domini espacial $x \in [-1.0, 1.0]$ i un domini temporal $t \in [0.0, 1.0]$. Les condicions inicials i de contorn es defineixen de la següent manera per simular l'obertura d'un front d'ona:

- **Condició inicial:** Una funció d'Heaviside $u(x, 0) = -1.0$ si $x < 0$, i $u(x, 0) = 1.0$ si $x > 0$.
- **Condicions de contorn:** Es mantenen fixos els extrems del domini, sent $u(-1, t) = -1.0$ i $u(1, t) = 1.0$.

Solució Analítica de Referència

L'equació de Burgers no viscosa amb aquestes condicions inicials té una solució analítica coneguda com a ona de rarefacció. Aquesta equació es pot resoldre pel mètode de les característiques (vegeu la secció 8.1 de l'apèndix per a la resolució completa). Així, la solució exacta, $u(x, t)$, es

pot definir per a qualsevol $t > 0$ separant el domini en tres zones:

$$u(x, t) = \begin{cases} -1 & \text{si } x \leq -t, \\ \frac{x}{t} & \text{si } -t < x < t, \\ 1 & \text{si } x \geq t. \end{cases} \quad (21)$$

Aquesta funció exacta, estructurada per trossos, ens permetrà obtenir els valors reals d'avaluació de forma instantània i vectoritzada sense dependre d'integradors numèrics continus.

Resultats de la PINN

Per capturar correctament les discontinuïtats geomètriques de la solució, s'ha mantingut una xarxa neuronal profunda estructurada en 5 capes ocultes de 40 neurones cadascuna. El nombre de capes i neurones per les equacions s'ha escollit arbitràriament, no s'ha fet un estudi, tot i que s'ha agafat més capes i neurones en comparació a la logística ja que és un sistema més complex. El conjunt de dades d'entrenament consisteix en 5000 punts de col·locació col·locats de forma aleatòria dins del domini espai-temporal, a més de 500 punts per a les condicions inicials i 500 més per a les fronteres.

S'ha atorgat un pes de 100 a les condicions inicials i de contorn per tal que el model prioritzi el compliment de l'EDP per tot el domini i aquestes tinguin més importància.

A la Figura 4 s'aprecia el mapa de colors de la predicció, on es visualitza l'obertura simètrica de l'ona de rarefacció des de l'origen.

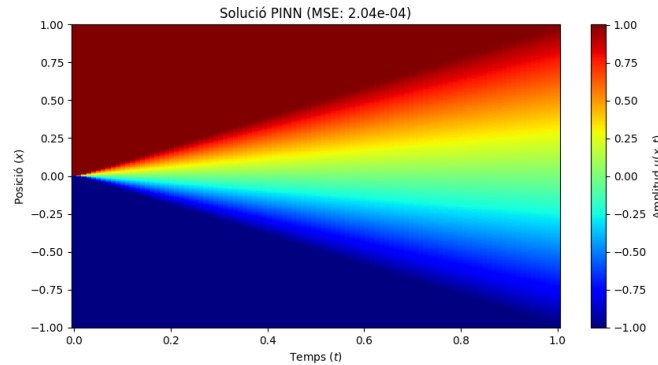


Figura 4: Solució predita per la PINN. S'observa la transició de l'esglaió inicial cap a una ona de rarefacció.

L'anàlisi de talls temporals (Figura 5, pàgina següent) mostra que la PINN s'apropa a la solució exacta: zones planes constants i un pendent lineal central (x/t).

5.3 Equació de Fisher

El tercer cas d'estudi és l'equació de Fisher, un cas particular dins de les equacions de reacció-difusió que combina l'evolució temporal, la difusió espacial i un terme de reacció logística.

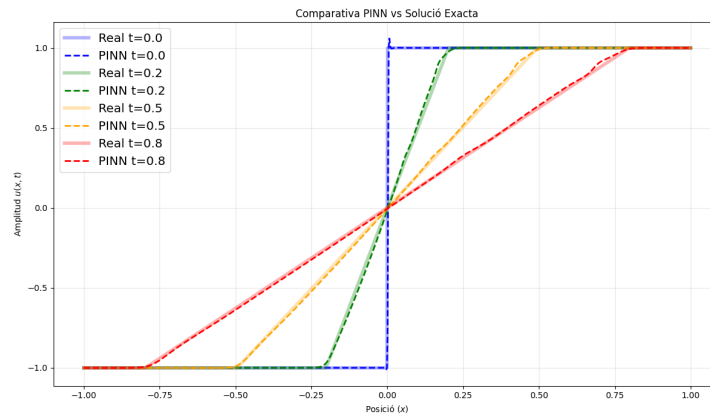


Figura 5: Comparativa entre la PINN i la solució analítica a $t = \{0.0, 0.2, 0.5, 0.8\}$ Per a l'equació de Burgers.

S'expressa com:

$$u_t - du_{xx} - au(1 - u) = 0, \quad (22)$$

on $u(x, t)$ és la variable d'estat, d és el coeficient de difusió i a representa la taxa de creixement del sistema.

Aquesta equació introdueix la difusió espacial, que estableix que el flux d'un material és proporcional al gradient de la seva concentració. Aquest fenomen es quantifica mitjançant el coeficient de difusió (d), el qual mesura l'eficiència amb què les partícules o poblacions es dispersen des de zones d'alta densitat cap a zones de baixa densitat.

Per a aquesta equació, s'ha considerat un domini espacial $x \in [-1.0, 1.0]$ i un domini temporal $t \in [0.0, 1.0]$, amb els paràmetres establerts a $d = 1$ i $a = 1$. Les condicions inicials i de contorn es defineixen de la següent manera:

- **Condició inicial:** Una funció d'Heaviside on $u(x, 0) = 1.0$ si $x \leq 0$, i $u(x, 0) = 0.0$ si $x > 0$.
- **Condicions de contorn:** Es mantenen fixos els extrems del domini, sent $u(-1, t) = 1.0$ i $u(1, t) = 0.0$.

Solució Numèrica de Referència

A diferència de l'equació logística o de l'equació de Burgers no viscosa, l'equació de Fisher subjecta a una condició inicial d'Heaviside no disposa d'una solució analítica exacta. Per tant, per poder validar l'error de la xarxa neuronal a qualsevol coordenada (x, t) s'ha de recórrer a mètodes d'integració numèrica tradicional.

Per obtenir aquesta matriu de referència, s'ha utilitzat la funció `solve_ivp` importada de la llibreria `scipy.integrate` de Python. Dins d'aquesta funció, s'ha definit l'integrador numèric especificant el paràmetre `method='BDF'`.

Resultats de la PINN

Per resoldre aquesta EDP, s'ha utilitzat una xarxa de 5 capes ocultes amb 40 neurones cadascuna. El conjunt de dades d'entrenament està format per 5000 punts de col·locació distribuïts de forma contínua per calcular el residu de l'equació, a més de 500 punts per a la condició inicial i 500 punts per a cadascuna de les fronteres.

Durant l'optimització, s'ha aplicat un pes de 100 a l'error de les condicions inicials i de contorn respecte a la pèrdua física, igual que en equacions anteriors.

A la Figura 6 es pot observar l'evolució espai-temporal predita per la PINN. El mapa de calor reflecteix correctament la relaxació de l'esglaó inicial en $t = 0$ provocada per la difusió.

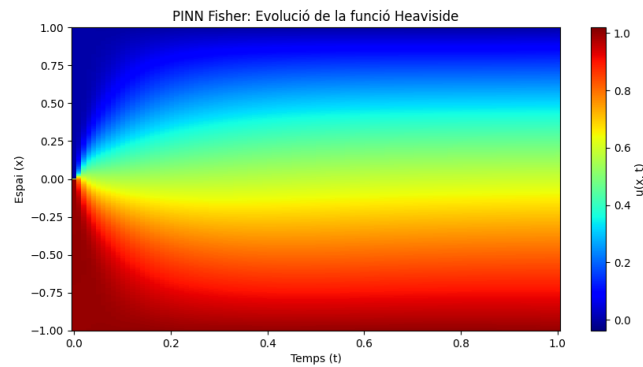


Figura 6: Mapa de color de la solució predita per la PINN. S'observa l'evolució i propagació del front d'ona generat a partir de la condició inicial d'Heaviside.

A la Figura 7 s'ha comparat la sortida de la xarxa amb una solució numèrica de referència.

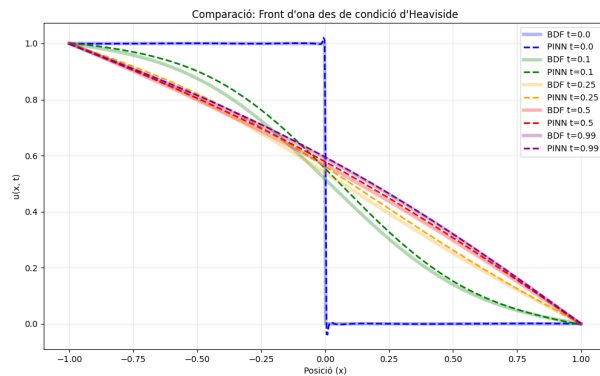


Figura 7: Comparació entre la predicció de la PINN i la solució numèrica en diferents instants de temps.

Tot i que es veu que hi ha una petita diferència entre el resultat de la PINN i el del mètode numèric, aquests són bastant propers.

6 La Inestabilitat de Turing

En el model anterior, hem vist com la difusió actua com una força estabilitzadora que tendeix a homogeneïtzar l'espai i suavitzar les discontinuïtats. En un sistema amb una única equació, si un equilibri és estable per a la reacció, ho segueix sent en afegir la difusió (consultar [2] per a la demostració). No obstant això, l'any 1952, el matemàtic Alan Turing va proposar una idea aparentment contraintuïtiva: la difusió pot desestabilitzar un sistema i conduir a la formació espontània de patrons espacials, un procés conegut com a morfogènesi química [6].

Turing va introduir aquest concepte originalment per donar resposta a l'aparició de patrons naturals, com ara les taques o les ratlles en el pelatge dels animals. Segons la seva proposta, la interacció i propagació d'uns determinats productes químics determina la pigmentació de la pell en funció del seu nivell de concentració.

6.1 Fonament Matemàtic

Definició del Sistema

Des d'un punt de vista matemàtic, la inestabilitat de Turing (o inestabilitat impulsada per la difusió) es defineix com aquell fenomen on un estat d'equilibri uniforme o homogeni, és a dir, que tampoc depèn de l'espai i, per tant, és constant, que és estable en absència de difusió (només amb el terme de reacció), esdevé inestable en afegir-hi la difusió.

Per tal que aquest trencament de la simetria tingui lloc i es formin patrons, no n'hi ha prou amb una sola equació, com s'ha comentat anteriorment. És necessari disposar d'un sistema format per, com a mínim, dues substàncies o poblacions que interaccionin i que tinguin coeficients de difusió diferents ($d_1 \neq d_2$).

Considerem un sistema de dues substàncies, u_1 i u_2 , en un domini espacial unidimensional. Les equacions reacció-difusió d'aquest sistema per a les densitats $u_1(x, t)$ i $u_2(x, t)$ en un domini espacial $x \in [0, L]$ s'expressen com:

$$\begin{cases} \frac{\partial u_1}{\partial t} = d_1 \frac{\partial^2 u_1}{\partial x^2} + f_1(u_1, u_2), \\ \frac{\partial u_2}{\partial t} = d_2 \frac{\partial^2 u_2}{\partial x^2} + f_2(u_1, u_2), \\ \frac{\partial u_1}{\partial x}(t, 0) = \frac{\partial u_1}{\partial x}(t, L) = 0, \\ \frac{\partial u_2}{\partial x}(t, 0) = \frac{\partial u_2}{\partial x}(t, L) = 0, \end{cases} \quad (23)$$

on d_1 i d_2 són els coeficients de difusió, i f_1 i f_2 són les funcions de reacció.

Suposem que existeix un equilibri homogeni $(\hat{u}_1, \hat{u}_2)$, és a dir, tal que $f_1(\hat{u}_1, \hat{u}_2) = 0$ i $f_2(\hat{u}_1, \hat{u}_2) = 0$.

Estabilitat sense Difusió

Per a que es produeixi la inestabilitat de Turing cal que, en absència de difusió ($d_1 = d_2 = 0$), $(\hat{u}_1, \hat{u}_2)$ sigui estable.

Per avaluar el comportament del sistema davant d'una petita pertorbació, definim una solució

propera a l'equilibri de la forma $u_1(x, t) = \hat{u}_1 + \tilde{u}_1(x, t)$ i $u_2(x, t) = \hat{u}_2 + \tilde{u}_2(x, t)$, on $\tilde{u}_1$ i $\tilde{u}_2$ són pertorbacions petites.

Atès que les funcions de reacció f_1 i f_2 són generalment no lineals, el sistema s'aproxima linealitzant-lo mitjançant l'expansió en sèrie de Taylor per a diverses variables al voltant del punt d'equilibri $(\hat{u}_1, \hat{u}_2)$:

$$\begin{aligned} f_1(u_1, u_2) &= f_1(\hat{u}_1, \hat{u}_2) + \frac{\partial f_1}{\partial u_1}(\hat{u}_1, \hat{u}_2)\tilde{u}_1 + \frac{\partial f_1}{\partial u_2}(\hat{u}_1, \hat{u}_2)\tilde{u}_2 + \dots, \\ f_2(u_1, u_2) &= f_2(\hat{u}_1, \hat{u}_2) + \frac{\partial f_2}{\partial u_1}(\hat{u}_1, \hat{u}_2)\tilde{u}_1 + \frac{\partial f_2}{\partial u_2}(\hat{u}_1, \hat{u}_2)\tilde{u}_2 + \dots \end{aligned} \quad (24)$$

Com que $(\hat{u}_1, \hat{u}_2)$ és l'equilibri homogeni, sabem per definició que $f_1(\hat{u}_1, \hat{u}_2) = 0$ i $f_2(\hat{u}_1, \hat{u}_2) = 0$. A més, com que assumim que les pertorbacions $\tilde{u}_1$ i $\tilde{u}_2$ són prou petites, els termes d'ordre superior de l'expansió esdevenen molt petits i s'ignoren i, per tant, el sistema es transforma en el següent sistema lineal:

$$\begin{cases} \frac{\partial \tilde{u}_1}{\partial t} \approx \frac{\partial f_1}{\partial u_1} \tilde{u}_1 + \frac{\partial f_1}{\partial u_2} \tilde{u}_2, \\ \frac{\partial \tilde{u}_2}{\partial t} \approx \frac{\partial f_2}{\partial u_1} \tilde{u}_1 + \frac{\partial f_2}{\partial u_2} \tilde{u}_2. \end{cases} \quad (25)$$

Això ens permet expressar el sistema linealitzat de forma matricial utilitzant la matriu Jacobiana J :

$$J = \begin{pmatrix} f_{1u_1} & f_{1u_2} \\ f_{2u_1} & f_{2u_2} \end{pmatrix}, \quad (26)$$

on cada terme f_{iu_j} denota la derivada parcial $\frac{\partial f_i}{\partial u_j}$ avaluada en el punt d'equilibri $(\hat{u}_1, \hat{u}_2)$.

Perquè l'equilibri sigui estable en aquest sistema linealitzat i, per tant, també en el sistema no lineal pel teorema de Hartman [4], els valors propis de la matriu Jacobiana J han de tenir la part real negativa. Aquests valors propis són les arrels del polinomi característic, el qual es pot expressar en funció de la traça ($\text{Tr}(J)$) i del determinant ($\det(J)$) de la matriu:

$$\lambda^2 - (f_{1u_1} + f_{2u_2})\lambda + (f_{1u_1}f_{2u_2} - f_{1u_2}f_{2u_1}) = \lambda^2 - \text{Tr}(J)\lambda + \det(J) = 0. \quad (27)$$

Les arrels d'aquesta equació, $\lambda_{1,2}$, venen donades per:

$$\lambda_{1,2} = \frac{\text{Tr}(J) \pm \sqrt{\text{Tr}(J)^2 - 4\det(J)}}{2}. \quad (28)$$

D'aquí s'obtenen les dues primeres condicions per a que es produeixi la inestabilitat de Turing:

- **Condició 1:** $\text{Tr}(J) = f_{1u_1} + f_{2u_2} < 0$.
- **Condició 2:** $\det(J) = f_{1u_1}f_{2u_2} - f_{1u_2}f_{2u_1} > 0$.

Inestabilitat amb Difusió

Ara tornem a considerar el sistema amb difusió (23), és a dir, $d_1, d_2 > 0$. Volem trobar les condicions per tal que l'equilibri homogeni, prèviament estable, es desestabilitzi davant de petites pertorbacions espacials.

Com abans, agafem petites pertorbacions de l'estat estacionari, $\tilde{u}_1(x, t) = u_1 - \hat{u}_1$ i $\tilde{u}_2(x, t) = u_2 - \hat{u}_2$, i substituïm aquestes en les equacions de (23). Com que $\hat{u}_1$ i $\hat{u}_2$ són constants (són l'equilibri homogeni), les seves derivades espacials i temporals són nul·les, de manera que el sistema en funció de les pertorbacions s'expressa com:

$$\begin{cases} \frac{\partial \tilde{u}_1}{\partial t} = d_1 \frac{\partial^2 \tilde{u}_1}{\partial x^2} + f_1(\hat{u}_1 + \tilde{u}_1, \hat{u}_2 + \tilde{u}_2), \\ \frac{\partial \tilde{u}_2}{\partial t} = d_2 \frac{\partial^2 \tilde{u}_2}{\partial x^2} + f_2(\hat{u}_1 + \tilde{u}_1, \hat{u}_2 + \tilde{u}_2). \end{cases} \quad (29)$$

Utilitzant el desenvolupament en sèrie de Taylor per linealitzar les funcions de reacció f_1 i f_2 al voltant del punt d'equilibri $(\hat{u}_1, \hat{u}_2)$ (24) i (25) i recordant que $f_i(\hat{u}_1, \hat{u}_2) = 0$, retenim només els termes d'ordre 1. Això ens permet expressar el sistema simplificat resultant en forma matricial:

$$\begin{pmatrix} \frac{\partial u_1}{\partial t} \\ \frac{\partial u_2}{\partial t} \end{pmatrix} = J \begin{pmatrix} u_1 \\ u_2 \end{pmatrix} + D \begin{pmatrix} \frac{\partial^2 u_1}{\partial x^2} \\ \frac{\partial^2 u_2}{\partial x^2} \end{pmatrix}, \quad (30)$$

on $D = \begin{pmatrix} d_1 & 0 \\ 0 & d_2 \end{pmatrix}$ és la matriu de coeficients de difusió.

Per resoldre aquesta equació en derivades parcials i estudiar l'evolució de les pertorbacions, s'aplica el mètode de separació de variables. Busquem una solució on la dependència temporal i l'espacial estiguin desacoblades, és a dir, $u(x, t) = T(t)X(x)$ (per a la resolució consultar [2]).

- **Part Temporal ($T(t)$):** La resolució de l'equació diferencial ordinària resultant per al temps condueix a una funció exponencial $e^{\sigma t}$, on σ és la taxa de creixement o valor propi.
- **Part Espacial ($X(x)$):** Assumint condicions de contorn de Neumann als extrems del domini (flux nul, $\frac{\partial u}{\partial x} = 0$), la solució de l'equació diferencial ordinària espacial adopta la forma d'una funció cosinus, $\cos(qx)$, on q és el nombre d'ona.

Per tant, proposem una solució pertorbada de (30) que pren la forma del terme general d'una expansió de Fourier:

$$\begin{pmatrix} u_1 \\ u_2 \end{pmatrix} = \begin{pmatrix} C_1 \\ C_2 \end{pmatrix} e^{\sigma t} \cos(qx). \quad (31)$$

Substituint aquesta solució proposada en el sistema linealitzat i calculant les derivades temporals i espacials, obtenim les següents expressions:

$$\begin{aligned} \begin{pmatrix} \frac{\partial u_1}{\partial t} \\ \frac{\partial u_2}{\partial t} \end{pmatrix} &= \sigma \begin{pmatrix} C_1 \\ C_2 \end{pmatrix} e^{\sigma t} \cos(qx) = \sigma \begin{pmatrix} u_1 \\ u_2 \end{pmatrix}, \\ \begin{pmatrix} \frac{\partial^2 u_1}{\partial x^2} \\ \frac{\partial^2 u_2}{\partial x^2} \end{pmatrix} &= -q^2 \begin{pmatrix} C_1 \\ C_2 \end{pmatrix} e^{\sigma t} \cos(qx) = -q^2 \begin{pmatrix} u_1 \\ u_2 \end{pmatrix}. \end{aligned} \quad (32)$$

Introduint aquestes derivades a l'equació matricial original (30), el sistema es transforma en:

$$\sigma \begin{pmatrix} u_1 \\ u_2 \end{pmatrix} = J \begin{pmatrix} u_1 \\ u_2 \end{pmatrix} - q^2 D \begin{pmatrix} u_1 \\ u_2 \end{pmatrix} \implies \sigma \begin{pmatrix} u_1 \\ u_2 \end{pmatrix} = (J - q^2 D) \begin{pmatrix} u_1 \\ u_2 \end{pmatrix}. \quad (33)$$

Si simplifiquem els termes espacials i temporals (equivalent a expressar-ho en funció de les constants C_1 i C_2), veiem que per obtenir solucions no trivials el vector de constants ha de ser un vector propi de la matriu $J - q^2 D$. El problema es redueix, doncs, a trobar els valors propis σ d'aquesta nova matriu que incorpora l'efecte de la difusió:

$$J - q^2 D = \begin{pmatrix} f_{1u_1} - q^2 d_1 & f_{1u_2} \\ f_{2u_1} & f_{2u_2} - q^2 d_2 \end{pmatrix}. \quad (34)$$

Per tenir inestabilitat de Turing, necessitem que la pertorbació creixi en el temps, la qual cosa implica que com a mínim un valor propi (σ) d'aquesta nova matriu ha de tenir la part real positiva ($\sigma > 0$).

Analitzem la traça i el determinant d'aquesta nova matriu (34):

- **La nova traça:** $\text{Tr}(J - q^2 D) = f_{1u_1} + f_{2u_2} - q^2(d_1 + d_2)$. Com que per la Condició 1 sabem que $f_{1u_1} + f_{2u_2} < 0$, i el terme $q^2(d_1 + d_2)$ és positiu, la nova traça sempre serà negativa.
- **El nou determinant:** Atès que la traça és negativa, l'única manera d'obtenir un valor propi amb part real positiva és que **el nou determinant esdevingui negatiu**. És a dir: $\det(J - q^2 D) < 0$.

Expressem aquest nou determinant com una funció de $z = q^2$ (on $z > 0$):

$$h(z) = d_1 d_2 z^2 - (d_2 f_{1u_1} + d_1 f_{2u_2})z + \det(J) < 0. \quad (35)$$

Aquesta funció $h(z)$ descriu una paràbola convexa (oberta cap a dalt). Perquè la paràbola prengui valors negatius, el seu vèrtex (el punt mínim) s'ha de situar per sota de l'eix horitzontal i en la regió on $z > 0$.

Calculem el mínim de la paràbola derivant $h(z)$ respecte a z i igualant a zero:

$$h'(z) = 2d_1 d_2 z - (d_2 f_{1u_1} + d_1 f_{2u_2}) = 0 \implies z_{\min} = \frac{d_2 f_{1u_1} + d_1 f_{2u_2}}{2d_1 d_2}. \quad (36)$$

Perquè aquest mínim correspongui a un nombre d'ona físicament possible ($z = q^2 > 0$), el numerador ha de ser positiu (ja que els coeficients de difusió d_1, d_2 ho són). Això ens proporciona una nova condició:

- **Condició 3:** $f_{1u_1} d_2 + f_{2u_2} d_1 > 0$.

Finalment, avaluem la funció en aquest punt mínim ($h(z_{\min}) < 0$) per garantir que el vèrtex

pren valors negatius. S'obté la següent desigualtat:

$$\frac{-(d_2 f_{1u_1} + d_1 f_{2u_2})^2}{4d_1 d_2} + \det(J) < 0. \quad (37)$$

Reordenant els termes per aïllar la condició final, obtenim l'última condició:

• **Condició 4:** $2\sqrt{d_1 d_2 \det(J)} < f_{1u_1} d_2 + f_{2u_2} d_1.$

Resum de les Condicions de Turing

Per tant, la inestabilitat de Turing es produeix només quan se satisfan simultàniament les quatre condicions següents:

1. $f_{1u_1} + f_{2u_2} < 0.$
2. $f_{1u_1} f_{2u_2} - f_{1u_2} f_{2u_1} > 0.$
3. $f_{1u_1} d_2 + f_{2u_2} d_1 > 0.$
4. $2\sqrt{d_1 d_2 \det(J)} < f_{1u_1} d_2 + f_{2u_2} d_1.$

6.2 Aplicació a un sistema concret: El Model de Meinhardt

Per posar en pràctica els conceptes d'inestabilitat de Turing i resoldre'ls posteriorment mitjançant una PINN, s'ha escollit el model de reacció-difusió de Meinhardt. Aquest és un sistema d'activador-inhibidor utilitzat per modelar la formació de patrons biològics complexos. El seu mecanisme es basa en una autoamplificació local i una inhibició de llarg abast: l'activador té una producció autocatalítica i una difusió lenta per generar pics de concentració, mentre que l'inhibidor frena aquesta producció i es difon molt més ràpidament per impedir la formació de nous centres d'activació a l'entorn [5].

A diferència del cas teòric unidimensional en l'espai vist anteriorment, ara treballarem en un domini espacial bidimensional. El sistema ve donat per les següents equacions en derivades parcials subjectes a condicions de contorn de Neumann (flux nul) al domini espacial $(x, y) \in [0, L]^2$:

$$\begin{cases} \frac{\partial u_1}{\partial t} = \alpha u_1^2 u_2 - \mu u_1 + d_1 \left(\frac{\partial^2 u_1}{\partial x^2} + \frac{\partial^2 u_1}{\partial y^2} \right), \\ \frac{\partial u_2}{\partial t} = \beta - \alpha u_1^2 u_2 + d_2 \left(\frac{\partial^2 u_2}{\partial x^2} + \frac{\partial^2 u_2}{\partial y^2} \right), \\ \frac{\partial u_1}{\partial x}(t, 0, y) = \frac{\partial u_1}{\partial x}(t, L, y) = \frac{\partial u_1}{\partial y}(t, x, 0) = \frac{\partial u_1}{\partial y}(t, x, L) = 0, \\ \frac{\partial u_2}{\partial x}(t, 0, y) = \frac{\partial u_2}{\partial x}(t, L, y) = \frac{\partial u_2}{\partial y}(t, x, 0) = \frac{\partial u_2}{\partial y}(t, x, L) = 0, \end{cases} \quad (38)$$

on $u_1(t, x, y)$ actua com a activador, $u_2(t, x, y)$ com a inhibidor, i els paràmetres $\alpha, \beta, \mu, d_1, d_2 > 0$ són constants estrictament positives.

Aplicant el que hem vist anteriorment, busquem primer l'equilibri homogeni $(\hat{u}_1, \hat{u}_2)$ on s'anul·len

les funcions de reacció locals:

$$\begin{cases} \alpha \hat{u}_1^2 \hat{u}_2 - \mu \hat{u}_1 = 0, \\ \beta - \alpha \hat{u}_1^2 \hat{u}_2 = 0, \end{cases} \implies (\hat{u}_1, \hat{u}_2) = \left(\frac{\beta}{\mu}, \frac{\mu^2}{\alpha\beta} \right). \quad (39)$$

A continuació, calculem la matriu Jacobiana J d'aquestes funcions de reacció avaluada exactament en aquest punt d'equilibri:

$$J = \begin{pmatrix} 2\alpha \hat{u}_1 \hat{u}_2 - \mu & \alpha \hat{u}_1^2 \\ -2\alpha \hat{u}_1 \hat{u}_2 & -\alpha \hat{u}_1^2 \end{pmatrix} = \begin{pmatrix} \mu & \frac{\alpha\beta^2}{\mu^2} \\ -2\mu & -\frac{\alpha\beta^2}{\mu^2} \end{pmatrix}. \quad (40)$$

Per tal que l'equilibri sigui estable en absència de difusió (Condicions 1 i 2), sabem que s'ha de complir que la traça sigui negativa

$$\text{Tr}(J) = \mu - \frac{\alpha\beta^2}{\mu^2} < 0, \quad (41)$$

i el determinant positiu

$$\det(J) = \frac{\alpha\beta^2}{\mu} > 0. \quad (42)$$

Ambdues es compleixen si definim els paràmetres de manera que $\mu^3 < \alpha\beta^2$.

Per avaluar el comportament de les pertorbacions espacials en aquest domini bidimensional subjecte a condicions de contorn de Neumann, escrivim el sistema linealitzat de forma vectorial afegint-hi els termes de difusió per a les dues dimensions:

$$\begin{pmatrix} \frac{\partial u_1}{\partial t} \\ \frac{\partial u_2}{\partial t} \end{pmatrix} = J \begin{pmatrix} u_1 \\ u_2 \end{pmatrix} + D \begin{pmatrix} \frac{\partial^2 u_1}{\partial x^2} \\ \frac{\partial^2 u_2}{\partial x^2} \end{pmatrix} + D \begin{pmatrix} \frac{\partial^2 u_1}{\partial y^2} \\ \frac{\partial^2 u_2}{\partial y^2} \end{pmatrix}, \quad (43)$$

on $D = \begin{pmatrix} d_1 & 0 \\ 0 & d_2 \end{pmatrix}$ representa la matriu dels coeficients de difusió.

La solució d'aquest sistema pren la forma següent:

$$\begin{pmatrix} u_1 \\ u_2 \end{pmatrix} = \begin{pmatrix} C_1 \\ C_2 \end{pmatrix} e^{\sigma t} \cos(q_1 x) \cos(q_2 y), \quad (44)$$

on, tal com s'obté de la resolució per separació de variables, els nombres d'ona per a cada dimensió són $q_1 = \frac{n\pi}{L}$ i $q_2 = \frac{m\pi}{L}$, sent $n, m \in \mathbb{N}_0$.

En tractar-se d'un cas amb dues dimensions espacials, definim el nombre d'ona al quadrat total com $q^2 = q_1^2 + q_2^2$. Per tal que tinguem inestabilitat de Turing, aquest valor ha de caure dins l'interval on el determinant de la matriu amb difusió és negatiu, és a dir, on la funció $h(q^2)$ definida a l'Equació 35 compleix $h(q^2) < 0$.

Per assegurar que aquest interval d'inestabilitat efectivament existeix, el sistema ha de complir les condicions 3 i 4 de Turing establertes teòricament. La tercera condició exigeix que $f_{1u_1}d_2 +$

$f_{2u_2}d_1 > 0$. Substituint-hi els termes de la nostra matriu Jacobiana:

$$\mu d_2 - \frac{\alpha\beta^2}{\mu^2}d_1 > 0 \implies \mu^3 d_2 - \alpha\beta^2 d_1 > 0. \quad (45)$$

La quarta condició demana que $2\sqrt{d_1 d_2 \det(J)} < f_{1u_1}d_2 + f_{2u_2}d_1$. Reescriuint-ho amb les nostres constants:

$$2\sqrt{\frac{d_1 d_2 \alpha \beta^2}{\mu}} < \mu d_2 - \frac{\alpha \beta^2 d_1}{\mu^2}. \quad (46)$$

Per tant, per tenir inestabilitats de Turing cal escollir unes constants que compleixin les quatre desigualtats (41), (42), (45) i (46). Establirem els paràmetres concrets del nostre problema com: $\alpha = 2.0$, $\beta = 1.0$, $\mu = 1.0$, $d_1 = 1.0$, $d_2 = 12.0$ i una mida de domini quadrat de costat $L = 25.0$. Aquests valors satisfan totes les condicions prèvies.

Primer cal trobar l'interval de valors de q^2 pels quals es té inestabilitat. Substituint les constants a la paràbola obtenim l'equació $h(z) = 12z^2 - 10z + 2 = 0$. Calculant-ne les arrels exactes, l'interval on la pertorbació creix resulta ser:

$$\frac{1}{3} < q^2 < \frac{1}{2}. \quad (47)$$

Com que sabem que $q_1 = \frac{n\pi}{L}$ i $q_2 = \frac{m\pi}{L}$, on $L = 25$ i $n, m \in \mathbb{N}_0$, podem reescriure l'interval anterior i trobar tots els valors de n i m que fan que es compleixi la inestabilitat:

$$\frac{1}{3} < \left(\frac{n\pi}{25}\right)^2 + \left(\frac{m\pi}{25}\right)^2 < \frac{1}{2}. \quad (48)$$

Aïllant l'expressió per trobar les combinacions d'enters de forma exacta, arribem a la següent inequació expressada amb fraccions:

$$\frac{25^2}{3\pi^2} < n^2 + m^2 < \frac{25^2}{2\pi^2}. \quad (49)$$

Si avaluem els extrems d'aquesta desigualtat numèricament (aproximadament $21.11 < n^2 + m^2 < 31.66$), podem deduir quins parells d'enters (n, m) cauen estrictament dins d'aquest domini. Els valors que ho compleixen són:

$$(n, m) \in \{(0, 5), (5, 0), (1, 5), (5, 1), (2, 5), (5, 2), (3, 4), (4, 3)\}.$$

6.2.1 Resultats de la PINN

Per resoldre aquest sistema d'EDPs i capturar correctament la formació de patrons espacials, s'ha utilitzat una xarxa de 5 capes ocultes amb 64 neurones cadascuna. Igual que abans, l'augment és degut a una complexitat major del problema i s'ha escollit 64 per a una millor eficiència durant l'entrenament. El conjunt de dades d'entrenament consisteix en 10000 punts de col·locació col·locats de forma aleatòria dins del domini espai-temporal, a més de 2000 punts per a les

condicions inicials i 2000 punts per a les fronteres. Durant l'optimització, s'ha aplicat un pes de 100 a l'error de la condició inicial i un pes de 10 a l'error de les condicions de contorn respecte a la pèrdua física.

En aquest cas, la condició inicial es defineix aplicant una petita pertorbació sobre l'estat d'equilibri homogeni del sistema, $(\hat{u}_1, \hat{u}_2) = \left(\frac{\beta}{\mu}, \frac{\mu^2}{\alpha\beta}\right)$, mitjançant les expressions següents:

$$\begin{cases} u_1(x, y, 0) = \hat{u}_1 + \epsilon \cos\left(\frac{n\pi x}{L}\right) \cos\left(\frac{m\pi y}{L}\right), \\ u_2(x, y, 0) = \hat{u}_2 + \epsilon \cos\left(\frac{n\pi x}{L}\right) \cos\left(\frac{m\pi y}{L}\right), \end{cases} \quad (50)$$

on $\epsilon = 0.1$ representa l'amplitud de la pertorbació, mentre que n i m corresponen als modes d'ona en les direccions de les coordenades x i y , respectivament. Com es pot observar en els mapes de color corresponents a l'instant inicial ($t = 0.0$), com que la pertorbació és la mateixa per a les dues substàncies, en el moment d'arrencada, ambdues presenten la mateixa distribució.

No obstant això, a mesura que el temps avança, els mapes de color reflecteixen correctament com la física del sistema pren el control i trenca aquesta paritat inicial. L'inhibidor, impulsat per un coeficient de difusió molt més elevat, canvia ràpidament la seva distribució de manera que ambdues substàncies generen ones totalment oposades: allà on l'activador assoleix el seu màxim de concentració, l'inhibidor hi presenta el seu mínim, i viceversa. Aquesta forma correspon al patró final.

S'ha provat el funcionament de les PINN per a dos valors de (n, m) diferents:

- **Evolució per al mode $(n = 5, m = 0)$:** Com que el nombre d'ona en la direcció y és 0, la inestabilitat només es desenvolupa al llarg de l'eix x . Això genera un patró de bandes o ratlles verticals paral·leles (vegeu la Figura 8). A les seqüències temporals es veu com l'inhibidor trenca el seu estat inicial per alinear les seves ratlles més fosques (mínims) amb les franges més lluminoses (màxims) de l'activador.

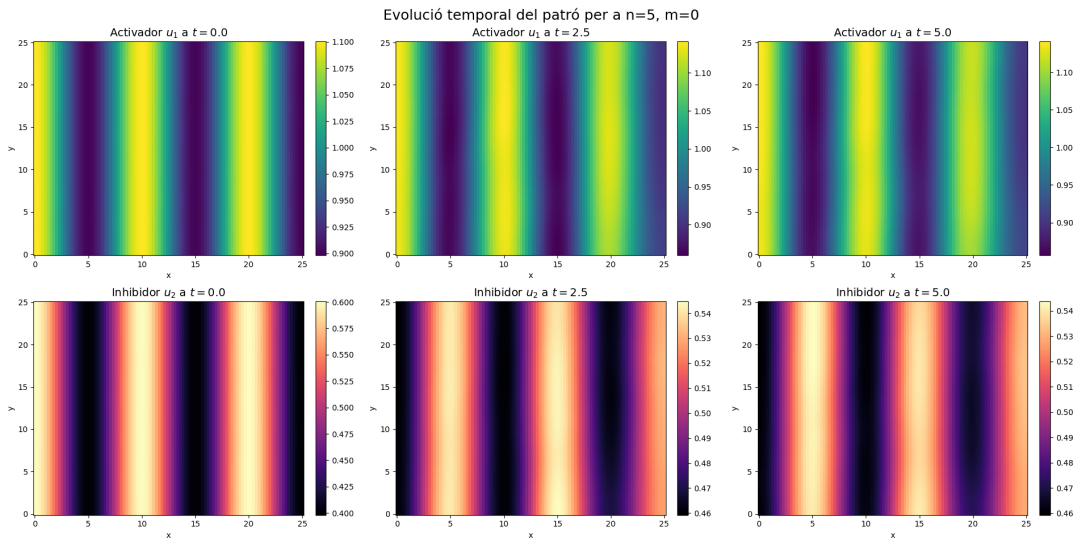


Figura 8: Evolució temporal del patró de bandes per al mode $(n = 5, m = 0)$. Es mostra la distribució espacial de l'activador (u_1) i de l'inhibidor (u_2) als instants $t = 0.0$, $t = 2.5$ i $t = 5.0$.

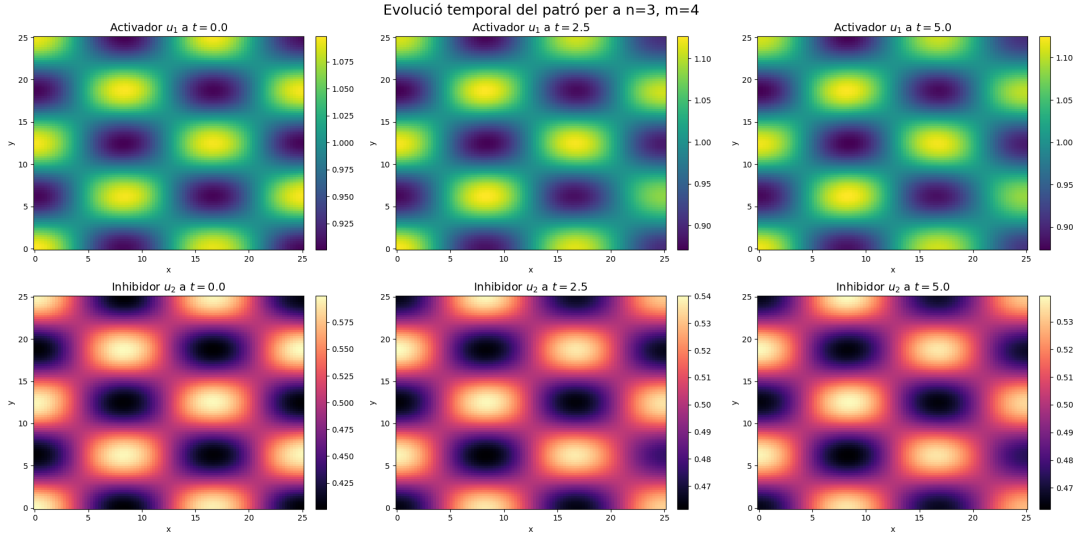


Figura 9: Evolució temporal del patró de taques (spots) per al mode $(n = 3, m = 4)$. Es mostra la distribució espacial de l'activador (u_1) i de l'inhibidor (u_2) als instants $t = 0.0$, $t = 2.5$ i $t = 5.0$.

- **Evolució per al mode $(n = 3, m = 4)$:** En introduir nombres d'ona no nuls en ambdues direccions espacials, s'origina un patró bidimensional en forma de taques o punts (vegeu la Figura 9). Igual que abans, l'inhibidor abandona l'estat inicial per a obtenir unes oscil·lacions oposades a l'activador.

Com hem vist, les PINNs són capaces de resoldre tant problemes senzills com l'equació logística o l'equació de Burgers, com problemes més complexos com l'equació de Meinhardt.

6.2.2 El problema invers en el model de Meinhardt

A diferència del problema directe, on l'objectiu és trobar l'evolució d'un sistema a partir d'una equació i uns paràmetres totalment coneguts, un problema invers consisteix a deduir quins són els paràmetres d'un sistema a partir d'observacions empíriques de la solució. Aquesta és una de les capacitats més potents de les PINNs, ja que permeten descobrir la dinàmica oculta d'un fenomen només aportant-hi dades de com aquest evoluciona.

Per a resoldre un problema invers, és a dir, permetre que el model pugui trobar els paràmetres, s'ha de modificar l'arquitectura de la PINN: els paràmetres físics de les equacions (α, β, μ, d_1 i d_2), abans constants, es redefeixen com a variables optimitzables del propi model. D'aquesta manera, durant l'entrenament, l'algorisme actualitza simultàniament els pesos de la xarxa i el valor d'aquests paràmetres.

Per resoldre aquest problema, idealment s'hauria d'entrenar la xarxa neuronal amb dades experimentals reals del sistema. No obstant això, atesa la dificultat d'obtenir aquestes mesures per al model de Meinhardt, s'ha generat un conjunt de dades sintètiques. Aquestes dades s'han simulat resolent l'equació original mitjançant un mètode numèric tradicional de diferències finites, extraient "fotografies" de la concentració de l'activador i l'inhibidor en diferents instants de temps per simular les lectures d'uns sensors.

Per avaluar la robustesa de la xarxa en aquesta tasca, s'han analitzat els mateixos dos patrons

d'inestabilitat que en el problema directe: el mode de bandes ($n = 5, m = 0$) i el mode de taques ($n = 3, m = 4$). Per a cadascun d'ells, s'han dissenyat tres experiments on s'amaguen diferents paràmetres de l'equació a la xarxa, obligant-la a descobrir-los simultàniament durant l'entrenament

Els experiments realitzats per a cada mode són:

1. **Cas 1:** Descobriment de les taxes de reacció (α i β).
2. **Cas 2:** Descobriment dels coeficients de difusió (d_1 i d_2).
3. **Cas 3:** Descobriment global de tots els paràmetres del sistema ($\alpha, \beta, \mu, d_1, d_2$).

Per dur a terme aquest descobriment de paràmetres, és necessari modificar l'arquitectura d'entrenament i, en especial, la funció de pèrdua de la xarxa neuronal. A diferència del problema directe, on l'error s'avaluava exclusivament a partir del residu de l'equació física i de les condicions inicials i de frontera, en el problema invers s'introdueix un nou terme: la pèrdua de dades o empírica ($\mathcal{L}_{data}$). Aquest component s'encarrega de minimitzar l'Error Quadràtic Mitjà (MSE) entre les concentracions predites pel model i els valors exactes provinents dels *snapshots* de les dades sintètiques als punts d'observació.

D'aquesta manera, la nova funció de pèrdua total que l'algorisme ha d'optimitzar queda definida com la suma ponderada dels diferents errors:

$$\mathcal{L}(\theta, \lambda) = w_{data}\mathcal{L}_{data}(\theta, \lambda) + w_{fisica}\mathcal{L}_{fisica}(\theta, \lambda) + w_{ic}\mathcal{L}_{ic}(\theta, \lambda) + w_{bc}\mathcal{L}_{bc}(\theta, \lambda), \quad (51)$$

on λ representa el conjunt de paràmetres físics desconeguts que la xarxa està aprenent de forma iterativa juntament amb els seus propis pesos θ . El nou terme d'error s'expressa com:

$$\mathcal{L}_{data}(\theta) = \frac{1}{N_{data}} \sum_{i=1}^{N_{data}} \left(|u_{1\theta}(x_i, y_i, t_i) - u_{1_i}^{exp}|^2 + |u_{2\theta}(x_i, y_i, t_i) - u_{2_i}^{exp}|^2 \right). \quad (52)$$

En aquesta expressió, N_{data} és el nombre de punts de les observacions proporcionades a la xarxa, u_{θ} corresponen a les solucions predites per la PINN, i u^{exp} són els valors objectiu del conjunt de dades sintètiques. Per tal d'assegurar que el model prioritzi l'ajust correcte als patrons geomètrics de les dades, durant la fase d'optimització s'ha aplicat un pes elevat de $w_{data} = 100$, mantenint alhora el pes de $w_{ic} = 100$ per a l'error de la condició inicial i de $w_{bc} = 10$ per a les condicions de contorn, i un pes unitari de 1 per a la pèrdua física.

A les taules 1 i 2 es presenten els valors predits per la PINN en comparació amb els valors teòrics reals utilitzats per generar les dades sintètiques, així com l'Error Quadràtic Mitjà (MSE) final de les distribucions espacials predites per a l'activador (u_1) i l'inhibidor (u_2).

Quan s'intenta descobrir només una part de la física (Cas 1 i Cas 2), la xarxa obté una bona precisió, identificant tant les taxes de reacció com els coeficients de difusió amb valors gairebé idèntics als teòrics i obtenint errors de predicció de l'ordre de 10^{-6} .

Experiment	Paràmetre	Valor Real	Predicció PINN	MSE de la Solució
Cas 1: Taxes de Reacció	α	2.0000	1.9909	$u_1: 1.55 \times 10^{-5}$
	β	1.0000	0.9956	$u_2: 3.19 \times 10^{-6}$
Cas 2: Coef. de Difusió	d_1	1.0000	1.0099	$u_1: 1.03 \times 10^{-5}$
	d_2	12.0000	12.0218	$u_2: 4.84 \times 10^{-6}$
Cas 3: Tots els paràmetres	α	2.0000	1.2071	$u_1: 1.30 \times 10^{-4}$
	β	1.0000	0.6024	$u_2: 5.32 \times 10^{-5}$
	μ	1.0000	0.6029	
	d_1	1.0000	0.5849	
	d_2	12.0000	7.0246	

Taula 1: Resultats del problema invers per al mode d'instabilitat de bandes ($n = 5, m = 0$).

Experiment	Paràmetre	Valor Real	Predicció PINN	MSE de la Solució
Cas 1: Taxes de Reacció	α	2.0000	1.9823	$u_1: 2.35 \times 10^{-5}$
	β	1.0000	1.0122	$u_2: 7.45 \times 10^{-6}$
Cas 2: Coef. de Difusió	d_1	1.0000	1.0104	$u_1: 2.37 \times 10^{-5}$
	d_2	12.0000	12.0254	$u_2: 6.33 \times 10^{-6}$
Cas 3: Tots els paràmetres	α	2.0000	1.2159	$u_1: 4.19 \times 10^{-5}$
	β	1.0000	0.6085	$u_2: 2.15 \times 10^{-5}$
	μ	1.0000	0.6088	
	d_1	1.0000	0.5884	
	d_2	12.0000	7.3232	

Taula 2: Resultats del problema invers per al mode d'instabilitat de taques ($n = 3, m = 4$).

No obstant això, en el Cas 3, on s'exigeix a la xarxa que descobreixi els cinc paràmetres simultàniament, s'observa un fenomen de manca d'identificabilitat (o no-unicitat de la solució). Les prediccions per a α, β, μ, d_1 i d_2 s'allunyen significativament dels valors teòrics, mostrant una mena d'escalat proporcional. Tot i això, el model assoleix un MSE molt baix, la qual cosa significa que la PINN ha trobat un conjunt de paràmetres diferent que produeix una dinàmica d'activació i inhibició molt similar de l'original. Aquest resultat posa de manifest que múltiples combinacions de paràmetres poden conduir a la formació d'un mateix patró.

7 Conclusions

Durant aquest treball s'ha pogut assolir una comprensió profunda de les Physics-Informed Neural Networks (PINNs) i del seu funcionament intern. S'ha explorat com l'ús de la diferenciació automàtica i la incorporació de les lleis físiques a la funció de pèrdua permeten a una xarxa neuronal aprendre la dinàmica d'un sistema. A partir de la implementació pràctica, s'ha validat l'eficàcia d'aquests models resolent des d'equacions clàssiques, fins a arribar a simular sistemes més complexos com la inestabilitat de Turing mitjançant el model bidimensional de Meinhardt. A més, s'ha demostrat una de les grans fortaleeses d'aquestes xarxes: la capacitat per resoldre problemes inversos i descobrir paràmetres físics ocults a partir de dades sintètiques.

Pel que fa als objectius proposats, el treball ha estat útil per posar en pràctica la connexió entre les equacions diferencials i la intel·ligència artificial. En la posada en pràctica, però, també s'han fet evidents les limitacions pràctiques que acompanyen aquests models. Sobretot, ha destacat el cost temporal requerit en la fase d'entrenament i la manca de garanties de convergència de l'optimitzador. Enfrontar-se a aquestes dificultats ha estat part del repte del projecte, ja que ha obligat a buscar solucions en la implementació per tal d'assegurar que el model pogués assolir els resultats desitjats.

Finalment, cal destacar que les PINNs són un camp d'estudi relativament recent. Aquest treball permet confirmar que tenen un gran potencial, però també deixa clar que encara hi ha molt per investigar. Queden força aspectes per millorar i optimitzar abans que aquesta tecnologia pugui aplicar-se i substituir els mètodes tradicionals. Sens dubte, és una àrea on caldrà seguir aprofundint en els propers anys per descobrir fins on poden arribar aquestes xarxes a l'hora de resoldre problemes físics complexos.

8 Apèndix

8.1 Resolució de l'equació de Burgers mitjançant el mètode de les característiques

Per obtenir la solució analítica de l'equació de Burgers no viscosa en el domini $x \in [-1, 1]$ i $t \in [0, 1]$:

$$u_t + uu_x = 0, \quad (53)$$

subjecta a la condició inicial

$$u(x, 0) = \begin{cases} -1 & \text{si } x < 0, \\ 1 & \text{si } x > 0, \end{cases} \quad (54)$$

ens basem en el mètode de les característiques [3], el qual permet reduir una equació en derivades parcials de primer ordre a un sistema apropiat d'equacions diferencials ordinàries.

Si escrivim l'equació de Burgers seguint la forma general quasilineal:

$$a(x, t, u)u_x + b(x, t, u)u_t = c(x, t, u), \quad (55)$$

identifiquem ràpidament que els coeficients de l'equació són $a = u$, $b = 1$ i $c = 0$.

Definim una corba inicial parametritzada per un paràmetre s , donada per $\Gamma = (x_0(s), t_0(s), u_0(s)) = (s, 0, u_0(s))$. En aquest problema, la funció $u_0(s)$ correspon a la condició inicial en forma de salt (funció d'Heaviside), que s'expressa explícitament de la següent manera:

$$u_0(s) = \begin{cases} -1 & \text{si } s < 0, \\ 1 & \text{si } s > 0. \end{cases} \quad (56)$$

Utilitzant un segon paràmetre τ per indicar la posició al llarg de la corba característica, obtenim l'anomenat sistema característic subjecte a les condicions inicials avaluades per a $\tau = 0$:

$$\begin{cases} \frac{dx}{d\tau} = u(\tau), \\ \frac{dt}{d\tau} = 1, \\ \frac{du}{d\tau} = 0, \end{cases} \quad \text{amb c.i.} \quad \begin{cases} x(0) = s, \\ t(0) = 0, \\ u(0) = u_0(s). \end{cases} \quad (57)$$

L'última equació diferencial del sistema (57), $\frac{du}{d\tau} = 0$, ens indica que la taxa instantània de canvi de u al llarg de la corba és nul·la, per tant la solució es manté constant sobre les rectes característiques, de manera que $u(\tau) = u_0(s)$. Coneixent això, podem integrar tot el sistema aplicant les condicions inicials corresponents per obtenir l'evolució de cada característica:

$$\begin{cases} t(\tau, s) = \tau, \\ u(\tau, s) = u_0(s), \\ x(\tau, s) = u_0(s)\tau + s. \end{cases} \quad (58)$$

Com que del sistema (58) sabem que $t = \tau$, podem substituir el paràmetre temporal a la coordenada espacial per trobar l'equació de les rectes característiques i aïllar el paràmetre inicial s en funció de les variables x i t :

$$x = u_0(s)t + s \implies s = x - u_0(s)t. \quad (59)$$

A partir d'aquesta relació general, avaluem el problema dividint el domini segons les dues regions que marca la nostra condició inicial $u_0(s)$ definida a (56):

- **Regió esquerra** ($s < 0$): La condició inicial ens marca que $u_0(s) = -1$. Substituint aquest valor a l'equació característica (59), obtenim $s = x + t$. Per tal que es compleixi la restricció $s < 0$, cal que $x + t < 0$, el que ens delimita la zona del domini $x < -t$. En aquesta regió, la solució constant és:

$$u(x, t) = -1. \quad (60)$$

- **Regió dreta** ($s > 0$): La condició inicial indica que $u_0(s) = 1$. Substituint aquest valor de nou a l'equació característica (59), resulta que $s = x - t$. Aplicant la restricció $s > 0$, cal que $x - t > 0$, delimitant la zona del domini $x > t$. Així, la solució constant és:

$$u(x, t) = 1. \quad (61)$$

El comportament d'aquestes dues regions ens indica que les rectes característiques divergeixen: les línies situades a $x < -t$ viatgen cap a l'esquerra, i les situades a $x > t$ avancen cap a la dreta. Aquest salt brusc de velocitats a l'origen de la corba inicial ($s = 0$) provoca que cap d'aquestes trajectòries entri a la regió central, generant així un buit al domini comprès entre $-t < x < t$.

Per tal de preservar la continuïtat física de la solució i omplir aquest buit d'informació, es genera l'anomenada ona de rarefacció. Aquest fenomen desplega un ventall continu de característiques que sorgeixen totes des del mateix origen inicial ($s = 0$, per tant $x_0 = 0$ i $t_0 = 0$) amb velocitats intermèdies $u \in (-1, 1)$. En aquesta zona, avaluant directament a la recta característica general de l'equació (59) per a $s = 0$, tenim:

$$x = u \cdot t \implies u(x, t) = \frac{x}{t}. \quad (62)$$

Així queda deduïda completament la funció definida a trossos per a qualsevol instant $t > 0$.

$$u(x, t) = \begin{cases} -1 & \text{si } x \leq -t, \\ \frac{x}{t} & \text{si } -t < x < t, \\ 1 & \text{si } x \geq t. \end{cases} \quad (63)$$

8.2 Codi

A continuació es presenten els fragments de codi estructurals de la implementació de les PINNs desenvolupada en PyTorch. Per consultar el codi font complet, es pot visitar el repositori de GitHub: <https://github.com/LlPanal/PINN>

Arquitectura del Model

Definició del model Perceptró Multicapa (MLP) utilitzant la tangent hiperbòlica com a funció d'activació.

```

1 import torch
2 import torch.nn as nn
3 import numpy as np
4
5 class PINN(nn.Module):
6     def __init__(self, input_dim=1, layers=3, neurons=20, output_dim=1):
7         super(PINN, self).__init__()
8
9         assert input_dim > 0, "La dimensio d'entrada ha de ser positiva"
10        assert layers >= 2, "El nombre de capes ha de ser almenys 2"
11        assert neurons > 0, "El nombre de neurones per capa ha de ser positiu"
12
13        layer_list = []
14        layer_list.append(nn.Linear(input_dim, neurons))
15        layer_list.append(nn.Tanh())
16        for _ in range(layers - 1):
17            layer_list.append(nn.Linear(neurons, neurons))
18            layer_list.append(nn.Tanh())
19        layer_list.append(nn.Linear(neurons, output_dim))
20
21        self.net = nn.Sequential(*layer_list)
22
23    def forward(self, t):
24        return self.net(t)

```

Listing 1: Codi en Python per a la definició de la xarxa neuronal PINN.

Funció d'Entrenament

Bucle principal d'optimització que calcula la pèrdua total i actualitza els pesos de la xarxa neuronal.

```

1 import torch
2
3 def train(
4     model,
5     physics_loss_fn,
6     boundary_loss_fn,
7     initial_loss_fn,
8     collocation_pts,
9     epochs=10000,
10    lr=1e-3,
11    weight_ic=100.0,
12    weight_bc=10.0
13 ):
14     optimizer = torch.optim.Adam(model.parameters(), lr=lr)
15
16     for epoch in range(epochs):
17         optimizer.zero_grad()
18
19         loss_ic = initial_loss_fn(model)
20         loss_bc = boundary_loss_fn(model)
21         loss_physics = physics_loss_fn(model, collocation_pts)
22
23         total_loss = weight_ic * loss_ic + weight_bc * loss_bc + loss_physics
24
25         total_loss.backward()
26         optimizer.step()

```

```

27
28         if epoch == 0 or (epoch + 1) % 100 == 0:
29             print(f"Epoch {epoch+1:05d} | Total: {total_loss.item():.8f} | "
30                   f"IC: {loss_ic.item():.8f} | BC: {loss_bc.item():.8f} | "
31                   f"Phys: {loss_physics.item():.8f}")
32
33     return model

```

Listing 2: Codi en Python per a l'algorisme d'entrenament i optimització de pèrdues.

Resolució del Model de Meinhardt

Implementació de les funcions de pèrdua específiques i execució de l'entrenament per al sistema de reacció-difusió de Meinhardt.

```

1  import torch
2  import numpy as np
3  import matplotlib.pyplot as plt
4  import os
5  from model import PINN
6  from train_2d import train
7
8  alpha = 2.0
9  beta = 1.0
10 mu = 1.0
11 d1 = 1.0
12 d2 = 12.0
13 L = 25.0
14 t_max = 5.0
15 model_path = "meinhardt_pinn.pth"
16
17
18 # Funcions de Perdua
19 def initial_loss_fn(model):
20     u_pred = model(xt_ic)
21     loss_u1 = torch.mean((u_pred[:, 0:1] - u1_target)**2)
22     loss_u2 = torch.mean((u_pred[:, 1:2] - u2_target)**2)
23     return loss_u1 + loss_u2
24
25 def boundary_loss_fn(model):
26     loss = 0
27     for axis, pt_list in bc_pts_dict.items():
28         idx = 0 if axis == 'x' else 1
29         for pts in pt_list:
30             u_pred = model(pts)
31             u1, u2 = u_pred[:, 0:1], u_pred[:, 1:2]
32
33             grad_u1 = torch.autograd.grad(u1, pts, torch.ones_like(u1), create_graph=True)
34             grad_u2 = torch.autograd.grad(u2, pts, torch.ones_like(u2), create_graph=True)
35
36             loss += torch.mean(grad_u1[:, idx:idx+1]**2) + torch.mean(grad_u2[:, idx:idx+1]**2)
37     return loss
38
39 def physics_loss_fn(model, collocation_pts):
40     u_pred = model(collocation_pts)
41     u1, u2 = u_pred[:, 0:1], u_pred[:, 1:2]
42
43     g1 = torch.autograd.grad(u1, collocation_pts, torch.ones_like(u1), create_graph=True)
44     u1_x, u1_y, u1_t = g1[:, 0:1], g1[:, 1:2], g1[:, 2:3]
45     u1_xx = torch.autograd.grad(u1_x, collocation_pts, torch.ones_like(u1_x),
46                                 create_graph=True)[0][:, 0:1]
47     u1_yy = torch.autograd.grad(u1_y, collocation_pts, torch.ones_like(u1_y),
48                                 create_graph=True)[0][:, 1:2]
49
50     g2 = torch.autograd.grad(u2, collocation_pts, torch.ones_like(u2), create_graph=True)
51     u2_x, u2_y, u2_t = g2[:, 0:1], g2[:, 1:2], g2[:, 2:3]
52     u2_xx = torch.autograd.grad(u2_x, collocation_pts, torch.ones_like(u2_x),
53                                 create_graph=True)[0][:, 0:1]

```

```

51     u2_yy = torch.autograd.grad(u2_y, collocation_pts, torch.ones_like(u2_y),
52                                 create_graph=True)[0][:, 1:2]
53
54     f1 = u1_t - (alpha * (u1**2) * u2 - mu * u1 + d1 * (u1_xx + u1_yy))
55     f2 = u2_t - (beta - alpha * (u1**2) * u2 + d2 * (u2_xx + u2_yy))
56     return torch.mean(f1**2) + torch.mean(f2**2)
57
58 if __name__ == "__main__":
59     model = PINN(input_dim=3, layers=layers, neurons=neurons, output_dim=2).to(device)
60     model = train(
61         model=model,
62         physics_loss_fn=physics_loss_fn,
63         boundary_loss_fn=boundary_loss_fn,
64         initial_loss_fn=initial_loss_fn,
65         collocation_pts=collocation_pts,
66         epochs=epochs,
67         lr=lr,
68         weight_ic=100.0,
69         weight_bc=10.0
70     )
71     torch.save(model.state_dict(), "meinhardt_pinn.pth")

```

Listing 3: Codi en Python del càlcul del residu físic i execució del model de Meinhardt.

Adaptacions per al Problema Invers

Per abordar el problema invers s'ha generat un conjunt de dades sintètiques que actuen com a observacions empíriques, i s'han fet modificacions clau als fitxers del model directe per permetre el descobriment de paràmetres.

Generació de dades sintètiques: Per obtenir les dades reals que el model haurà d'imitar, se simula l'equació utilitzant el mètode de diferències finites.

```

1  for step in range(steps):
2      if idx_guardat < len(instants_a_guardar) and t >= instants_a_guardar[idx_guardat]:
3          snapshot_data = np.column_stack((X.flatten(), Y.flatten(),
4                                             np.full_like(X.flatten(), t),
5                                             u1.flatten(), u2.flatten()))
6          dades_guardades.append(snapshot_data)
7          idx_guardat += 1
8
9      # Diferències Finites per al Laplacian i actualització de l'estat
10     lap_u1 = (u1[:-2, 1:-1] + u1[2:, 1:-1] + u1[1:-1, :-2] + u1[1:-1, 2:] - 4*u1[1:-1,
11     1:-1]) / (dx**2)
12     lap_u2 = (u2[:-2, 1:-1] + u2[2:, 1:-1] + u2[1:-1, :-2] + u2[1:-1, 2:] - 4*u2[1:-1,
13     1:-1]) / (dx**2)
14
15     react_u1 = alpha * (u1[1:-1, 1:-1]**2) * u2[1:-1, 1:-1] - mu * u1[1:-1, 1:-1]
16     react_u2 = beta - alpha * (u1[1:-1, 1:-1]**2) * u2[1:-1, 1:-1]
17
18     u1[1:-1, 1:-1] += dt * (d1 * lap_u1 + react_u1)
19     u2[1:-1, 1:-1] += dt * (d2 * lap_u2 + react_u2)

```

Listing 4: Integració numèrica per a la generació de dades sintètiques.

Modificació del model: Les constants de l'equació passen a ser variables optimitzables (`nn.Parameter`) si s'han d'aprendre, o es fixen com a `buffer` si ja són conegudes.

```

1     self.net = nn.Sequential(*layer_list)
2     all_param_names = ['alpha', 'beta', 'mu', 'd1', 'd2']
3
4     for name in all_param_names:
5         if name in params_to_learn:
6             init_val = params_to_learn[name]
7             setattr(self, name, nn.Parameter(torch.tensor([init_val], dtype=torch.
8             float32)))
9         else:

```

```

9         val = fixed_params.get(name, 1.0)
10        self.register_buffer(name, torch.tensor([val], dtype=torch.float32))

```

Listing 5: Configuració dels paràmetres interns del model.

Integració a l'entrenament: El bucle d'entrenament incorpora la nova pèrdua.

```

1    loss_data = data_loss_fn(model)
2    loss_physics = physics_loss_fn(model, collocation_pts)
3    loss_bc = boundary_loss_fn(model)
4    loss_ic = initial_loss_fn(model)
5
6    total_loss = (w_data * loss_data) + (w_physics * loss_physics) + \
7                (w_bc * loss_bc) + (w_ic * loss_ic)
8
9    total_loss.backward()
10   optimizer.step()

```

Listing 6: Càlcul de la pèrdua total en l'entrenament invers.

Funció de pèrdua empírica (meinhardt.py): Es carreguen els punts generats prèviament per crear els tensors d'entrenament i s'afegeix una nova funció que calcula l'error quadràtic mitjà respecte a aquestes dades.

```

1    dataset = np.load(data_path)
2    idx_aleatoris = np.random.choice(dataset.shape[0], n_data, replace=False)
3    dades_entrenament = dataset[idx_aleatoris, :]
4
5    xyt_data = torch.tensor(dades_entrenament[:, 0:3], dtype=torch.float32, device=device)
6    u_target = torch.tensor(dades_entrenament[:, 3:5], dtype=torch.float32, device=device)
7
8    def data_loss_fn(model):
9        u_pred = model(xyt_data)
10       return torch.mean((u_pred - u_target)**2)

```

Listing 7: Funció de pèrdua basada en les observacions experimentals.

Referències

- [1] Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). *Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations*. Journal of Computational Physics.
- [2] Diekmann, O., & Planqué, B. (2014). *Mathematical methods to gain biological insight*.
- [3] Pinchover, Y., & Rubinstein, J. (2005). *An Introduction to Partial Differential Equations*. Cambridge University.
- [4] Hartman, P. (1960). *A lemma in the theory of structural stability of differential equations*. Proceedings of the American Mathematical Society, 11(4), 610-620.
- [5] Meinhardt, H. (1982). *Models of biological pattern formation*. Academic Press, London.
- [6] Turing, A. M. (1952). *The chemical Basis of Morphogenesis*. Philosophical Transactions of the Royal Society of London. Series B, Biological Sciences, Vol.237, Pàg. 37-72. University of Manchester.
- [7] Edelstein-Keshet, L. (2005). *Mathematical Models in Biology*. Society for Industrial and Applied Mathematics (SIAM).
- [8] MATHWORKS. *¿Qué son las redes neuronales informadas por la física?* <https://es.mathworks.com/discovery/physics-informed-neural-networks.html>.
- [9] Henderson, I. (2022). *Physics Informed Neural Networks (PINNs): An Intuitive Guide*. Towards Data Science. <https://towardsdatascience.com/physics-informed-neural-networks-pinns-an-intuitive-guide>.
- [10] Kho, J., Wong, J. C., Chiu, P.-H., Koh, W., & Ooi, C. C. (2022). *Design of Turing Systems with Physics-Informed Neural Networks*.
- [11] Nielsen, M. A. (2015). *Neural Networks and Deep Learning*. <http://neuralnetworksanddeeplearning.com/>.
- [12] Markidis, S. (2021). *The Old and the New: Can Physics-Informed Deep-Learning Replace Traditional Linear Solvers?*.
- [13] Grossmann, T. G., Komorowska, U. J., Latz, J., & Schönlieb, C.-B. (2024). *Can physics-informed neural networks beat the finite element method?* IMA Journal of Applied Mathematics, 89(1), 143-174. <https://academic.oup.com/imamat/article/89/1/143/7680268>.